



Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Remove all adjacent duplicates in string

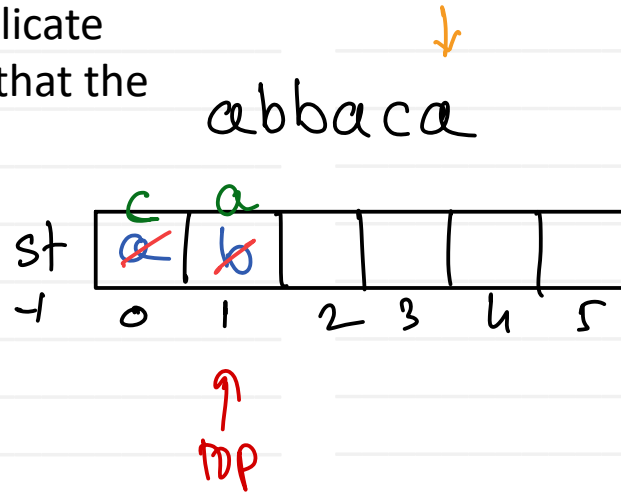
You are given a string s consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters and removing them.

We repeatedly make duplicate removals on s until we no longer can.

Return the final string after all such duplicate removals have been made. It can be proven that the answer is unique.

Example 1:
Input: $s = \text{"abbaca"}$
Output: "ca"

Example 2:
Input: $s = \text{"azxxzy"}$
Output: "ay"



```
string removeDuplicates(String s){
    int n = s.length();
    char st[] = new char[n];
    int top = -1;
    for(int i=0; i<n; i++){
        char curr = s.charAt(i);
        if( top != -1 && curr == st[top] )
            top--;
        else
            st[++top] = curr;
    }
    return new String(st, 0, top+1);
}
```

] == [

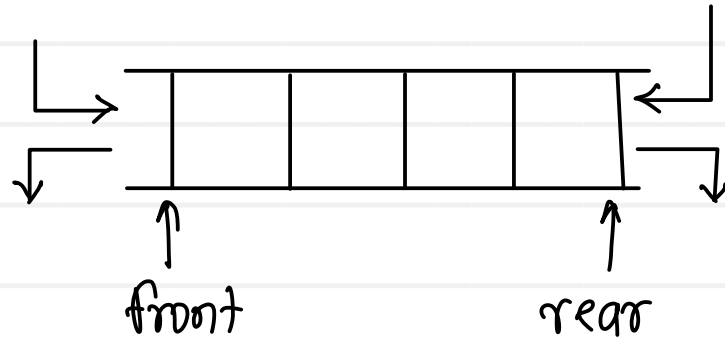
opening

(	[	{	<
0	1	2	3

closing

)	]	}	>
0	1	2	3

- data can be inserted or removed in a queue from both ends

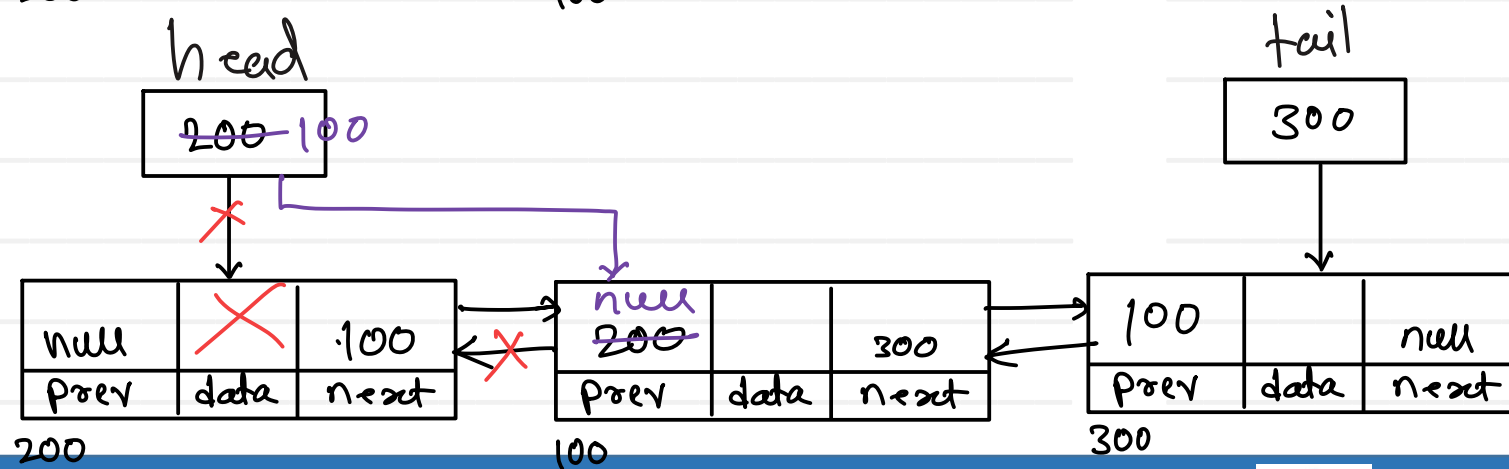
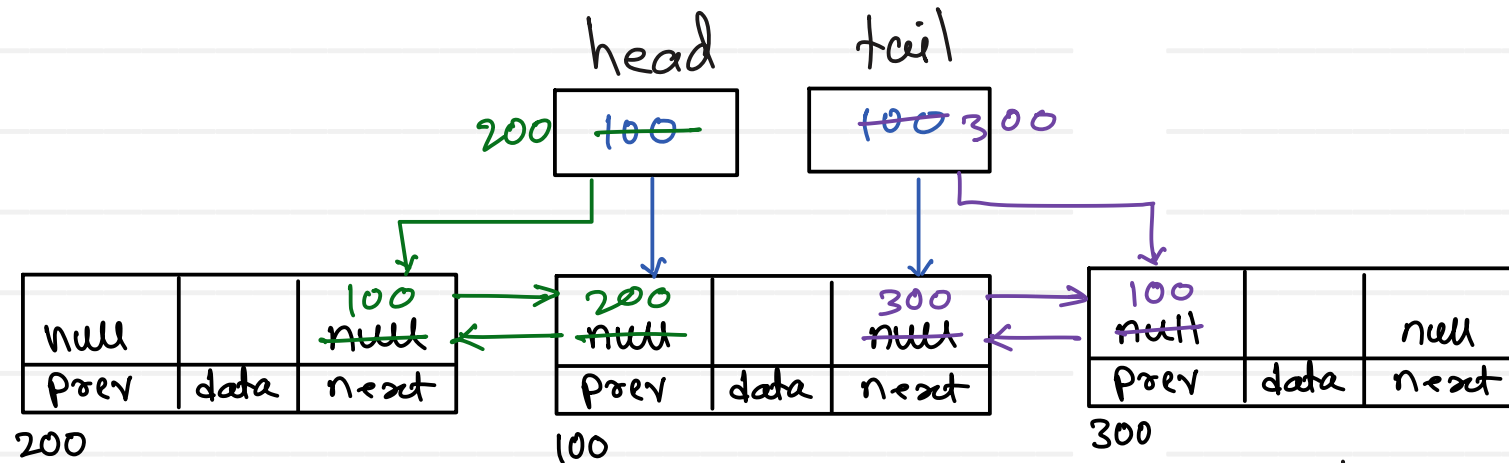


- Deque do not follow "FIFO" principle
- to implement double ended queue, we can use circular array or linked list

Operations :

1. pushFront()
2. popFront()
3. peekFront()
4. pushRear()
5. popRear()
6. peekRear()
7. isEmpty()
8. isFull()

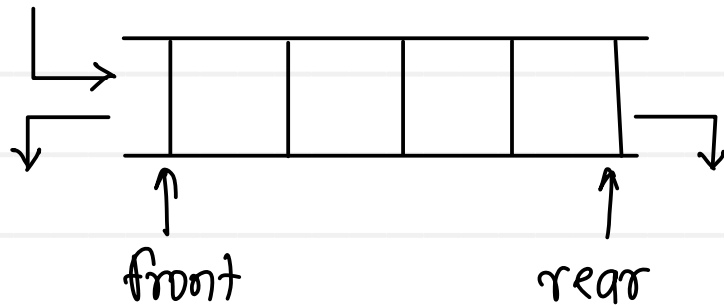
head $\rightarrow$ front : add first / delete first
 tail $\rightarrow$ rear : add last / delete last



Input Restricted Deque

remove from both ends

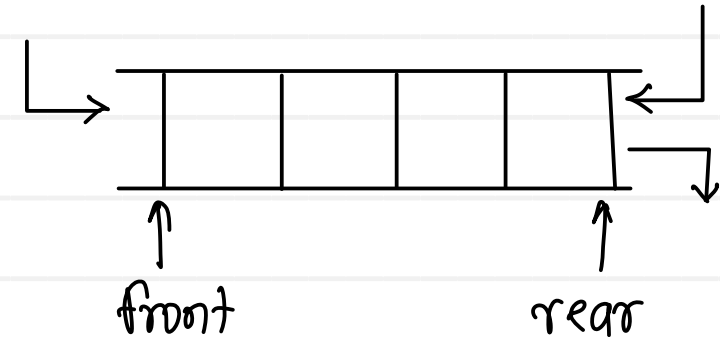
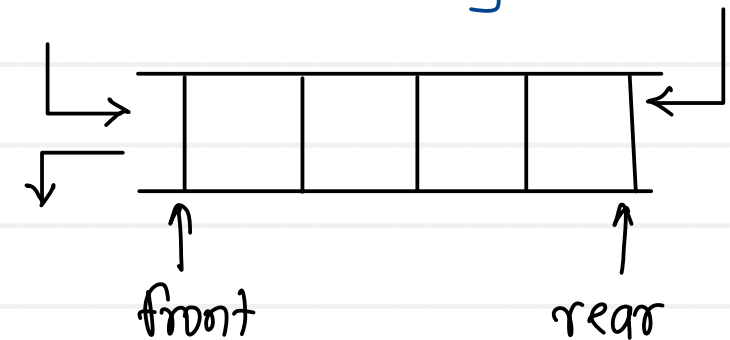
insert from only one end



Output Restricted Deque

insert from both ends

remove from only one end



- Dequeue is used to check palindrome efficiently than stack

head
↓
10 → 20 → 20 → 10

deque

	10	20	20	10	
--	----	----	----	----	--

f r

to push - n
to pop - n/2

$\frac{3}{2}n$

$T \propto \frac{3}{2}n$
 $T(n) = O(n)$
 $S(n) = O(n)$

- Deque is used to perform undo & redo operations efficiently



Deque

$op1 \rightarrow op2 \rightarrow op3 \rightarrow op4$

$\uparrow \qquad \qquad \qquad \uparrow$

$f \qquad \qquad \qquad r$

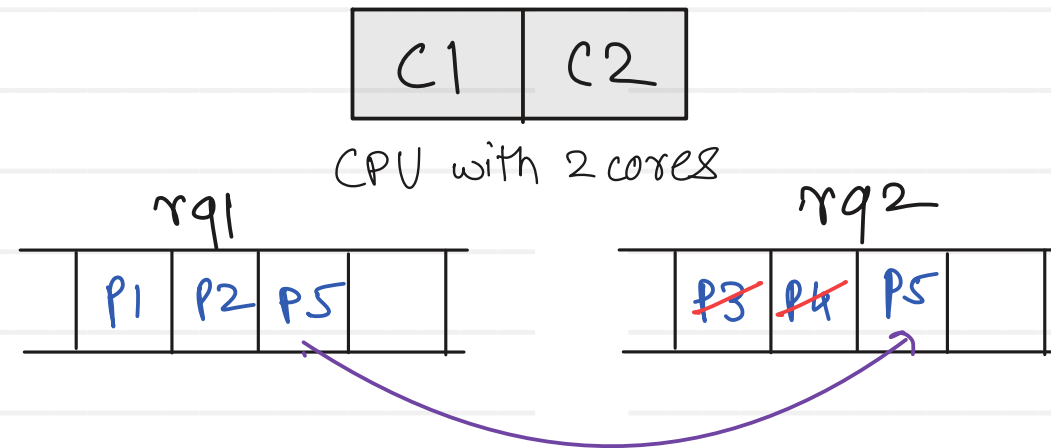
Undo

$op_3 \rightarrow op_4 \rightarrow op_1 \rightarrow op_2$

Redo

$op_1 \rightarrow op_2 \rightarrow op_3 \rightarrow op_4$

- deque is used to maintain browser history with some limit.
if limit exceeds history url is removed from another end
- deque is used in A-steal algorithm in OS



Linear search

1. decide/take key from user
2. traverse collection of data from one end to another
3. compare key with data of collection
 - 3.1 if key is matching
return index/true
 - 3.2 if key is not matching
return -1/false

88	33	66	99	11	77	22	55	14
0	1	2	3	4	5	6	7	8

$key == arr[i]$

77
Key

$i = 0, 1, 2, 3, 4, 5$
key is found

89
Key

$i = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9$
key is not found

88	33	66	99	11	77	22	55	14
0	1	2	3	4	5	6	7	8

Key = 88 $\rightarrow$ Best case : $O(1)$

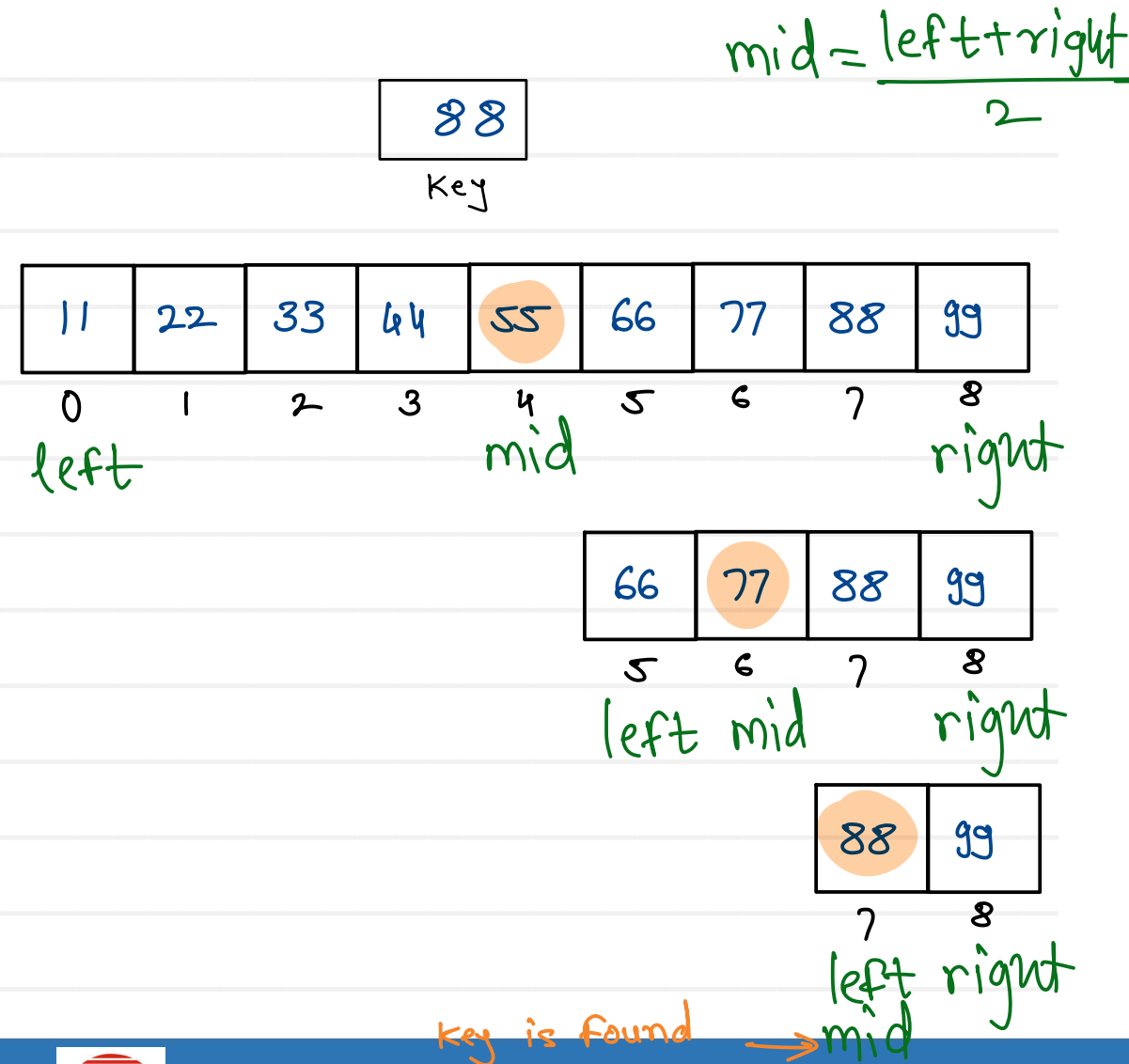
Key = 11 $\rightarrow$ Avg case : $O(n)$

Key = 14 or 89 $\rightarrow$ Worst case : $O(n)$

$$S(n) = O(1)$$

Binary search

1. take key from user
2. divide array into two parts
(find middle element)
3. compare middle element with key
 - 3.1 if key is matching
return index (mid)
 - 3.2 if key is less than middle element
search key in left partition
 - 3.3 if key is greater than middle element
search key in right partition
 - 3.4 if key is not matching
return -1



Binary search

25

key

11	22	33	44	55	66	77	88	99
----	----	----	----	----	----	----	----	----

0

1

2

3

4

5

6

7

8

left

mid

right

11	22	33	44
----	----	----	----

0

1

2

3

left

mid

right

33	44
----	----

2

1

1

2

2

3

left

right

mid

left partition : left to mid-1
right partition : mid+1 to right

valid partition : left $\leq$ right
invalid partition : left $>$ right

```

l = 0, r = 8, m;
while (l <= r) {
    m = (l + r) / 2;
    if (key == arr[m])
        return m;
    else if (key < arr[m])
        right = m - 1;
    else
        left = m + 1;
}
return -1;

```

11	22	33	44	55	66	77	88	99
0	1	2	3	4	5	6	7	8
key = 88								
l	r	l <= r				m		
0	8	T				4		
5	8	T				6		
7	8	T				7 ← key is found		
key = 25								
l	r	l <= r				m		
0	8	T				4		
0	3	T				1		
2	3	T				2		
2	1	F				← key is not found		

25
key

11	22	33	44	55	66	77	88	99
0	1	2	3	4	5	6	7	8
l				m				r

11	22	33	44
0	1	2	3
l	m		r

2	1	33	44
l	r	2	3
		l	r
		m	

main()

↓ ↑ -1

binarySearch(arr, 25, 0, 8) m = 4

↙ ↘ -1

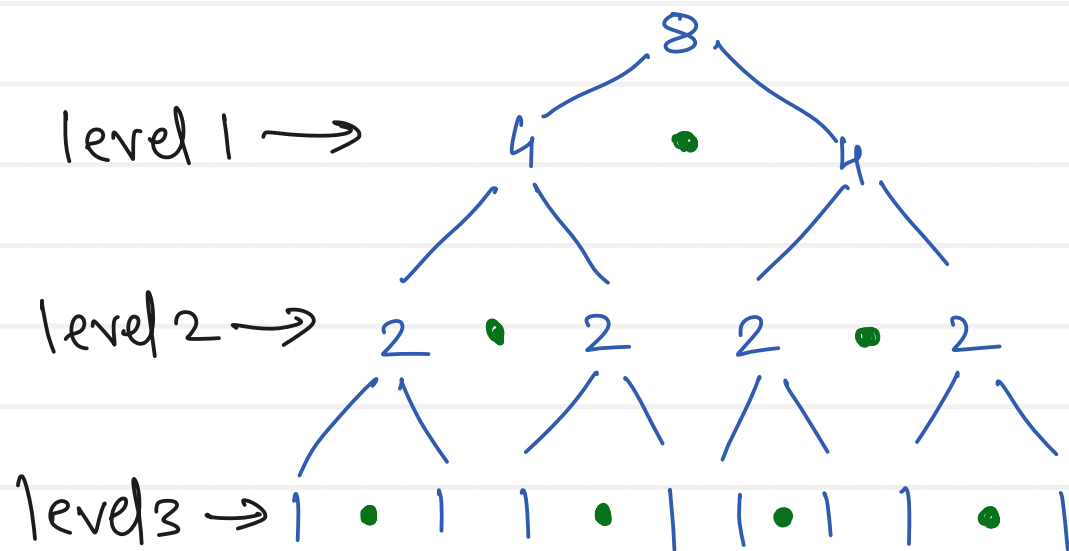
binarySearch(arr, 25, 0, 3) m = 1

↙ ↘ -1

binarySearch(arr, 25, 2, 3) m = 2

↙ ↘ -1

binarySearch(arr, 25, 2, 1)



n - no. of elements

l - no. of levels

$$2^l = n$$

$$2^l = n$$

$$\log 2^l = \log n$$

$$l \log 2 = \log n$$

$$l = \frac{\log n}{\log 2}$$

itrs = levels

$$\text{itrs} = \frac{\log n}{\log 2}$$

$$\text{Time} \propto \frac{\log n}{\log 2}$$

Arg
Worst

$$T(n) = O(\log n)$$

$O(1) \rightarrow$ best case

Searching algorithms analysis

- Time is directly proportional to number of comparisons
- For searching and sorting algorithms, count number of comparisons done

1. Linear search

- Best case - if key is found at few initial locations $\rightarrow O(1)$
- Average case - if key is found at middle locations $\rightarrow O(n)$
- Worst case - if key is found at last few locations/
key is not found $\rightarrow O(n)$

$S(n) = O(1)$

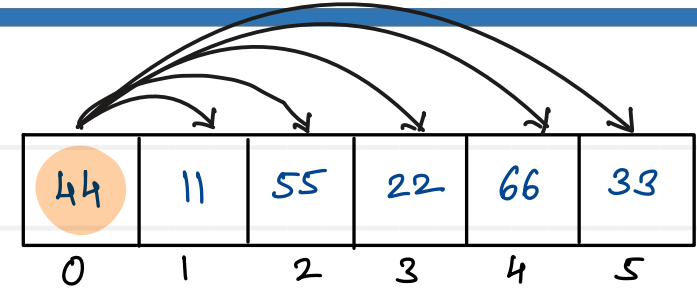
2. Binary search

- Best case - if key is found at first few levels $\rightarrow O(1)$
- Average case - if key is found at middle levels $\rightarrow O(\log n)$
- Worst case - if key is found at last level/
not found $\rightarrow O(\log n)$

$S(n) = O(1)$

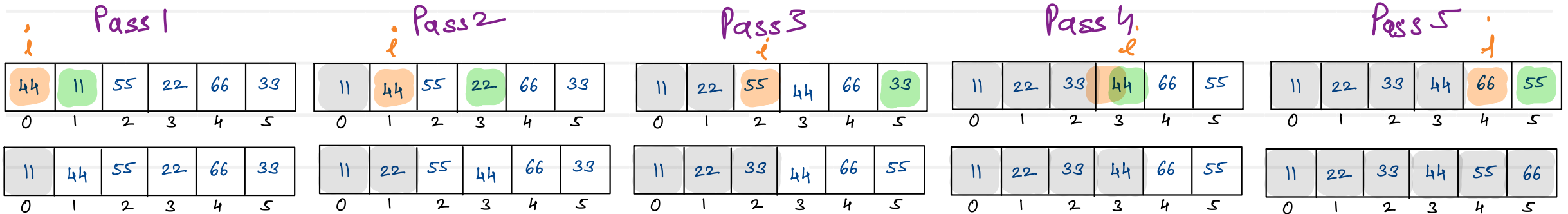
Selection sort

1. Select one position of the array
2. Find smallest element out of remaining elements
3. Swap selected position element and smallest element
4. Repeat above steps until array is sorted ($N-1$)



44	11	55	22	66	33
0	1	2	3	4	5

i $minIndex$ j
 3 3 4
 5
 6~~X~~



to select positions one by one : $i = 0$ to $N-2$ ($i < N-1$)
 to find smallest out of remaining : $j = i+1$ to $N-1$ ($j < N$)

Selection sort

44	11	55	22	66	33
0	1	2	3	4	5

i	minIndex	j
0	0	1
	1	2
		3
		4
		5
		6X

```

minIndex = i
for (j = i + 1; j < N; j++)
    if (arr[j] < arr[minIndex])
        minIndex = j;
    
```

11	44	55	22	66	33
0	1	2	3	4	5

i	minIndex	j
1	1	2
	2	3
		4
		5
		6X